

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I C.C.Jyoshna (192411098)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: C.c.Jyoshna

Q1. Refactor the C/C++ implementation of the final digest assembly and hexadecimal encoding routine

Introduction

The final digest assembly routine combines the generated digest words and converts them into hexadecimal characters. If the implementation repeatedly appends characters to a dynamically growing string, unnecessary copying and memory allocation can increase the execution time.

Problem Identified

A common inefficient implementation may process each byte and repeatedly concatenate hexadecimal strings. For example, repeatedly using string concatenation inside a loop can cause previously generated characters to be copied again and again.

Proposed Optimization

- The routine can be improved by:
- Allocating the required output buffer once.
- Writing hexadecimal characters directly into the buffer.
- Using a fixed lookup table such as 0123456789abcdef.
- Avoiding repeated string concatenation and unnecessary temporary strings.

Complexity Analysis

If repeated string concatenation causes the existing output to be copied for every new character:

Original Time Complexity: ($O(n^2)$)

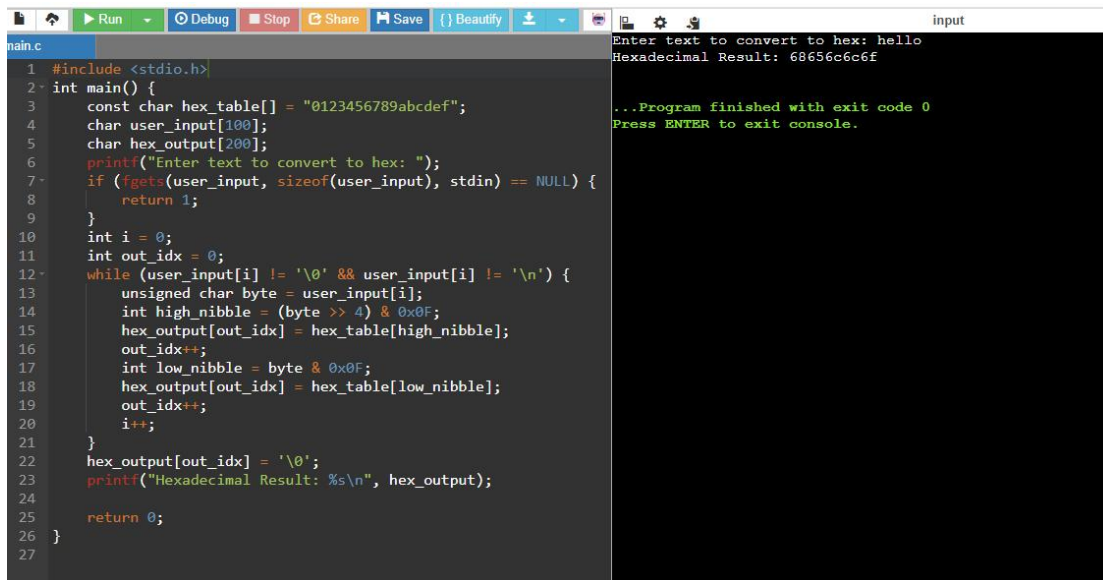
Improved Time Complexity: ($O(n)$)

where (n) is the number of digest characters.

Why is the Optimized Version Correct?

Each input byte is still converted into exactly two hexadecimal characters using the same hexadecimal mapping. The optimization only changes **how the output is stored**, not how the digest value is calculated.

Therefore, the optimized routine produces the **same hexadecimal digest** as the original implementation while requiring less execution time.



```
1 #include <stdio.h>
2 int main() {
3     const char hex_table[] = "0123456789abcdef";
4     char user_input[100];
5     char hex_output[200];
6     printf("Enter text to convert to hex: ");
7     if (fgets(user_input, sizeof(user_input), stdin) == NULL) {
8         return 1;
9     }
10    int i = 0;
11    int out_idx = 0;
12    while (user_input[i] != '\0' && user_input[i] != '\n') {
13        unsigned char byte = user_input[i];
14        int high_nibble = (byte >> 4) & 0x0F;
15        hex_output[out_idx] = hex_table[high_nibble];
16        out_idx++;
17        int low_nibble = byte & 0x0F;
18        hex_output[out_idx] = hex_table[low_nibble];
19        out_idx++;
20        i++;
21    }
22    hex_output[out_idx] = '\0';
23    printf("Hexadecimal Result: %s\n", hex_output);
24
25    return 0;
26 }
27
```

Enter text to convert to hex: hello
Hexadecimal Result: 68656c6c6f

...Program finished with exit code 0
Press ENTER to exit console.

Q2. Optimize the C/C++ implementation of the final digest assembly and hexadecimal encoding routine to minimize its space complexity while preserving correctness.

Introduction

The program stores the input and the complete hexadecimal result in separate arrays. This increases the memory used by the program, especially when the input becomes large.

Memory Hotspot

The main memory hotspot is: `char hex_output[200];`

This reserves 200 bytes even if the actual hexadecimal result is much smaller.

The `user_input[100]` buffer also occupies memory.

Proposed Optimization

Instead of creating a large fixed-size output array, the program can:

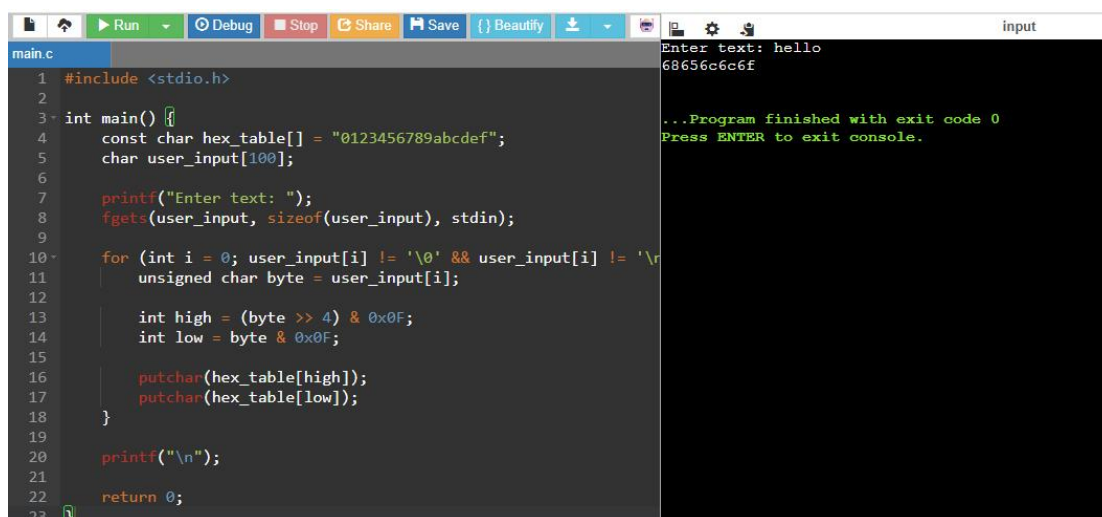
- Allocate only the required output space.
- Avoid unnecessary temporary buffers.
- Use the hexadecimal lookup table directly.
- If the hexadecimal result does not need to be stored, write the converted characters directly to the output using `putchar()`.

Space Complexity

- **Original:** $O(n)$, with fixed buffers that may allocate more memory than required.
- **Optimized:** $O(n)$ if the complete hexadecimal result must be stored, but with a much smaller actual memory footprint.
- If the result is **printed directly without storing it**, the additional output-buffer space becomes $O(1)$.

Conclusion

The main memory hotspot is the unnecessarily large hex_output buffer. Allocating only the required space, or directly printing the hexadecimal characters when storage is unnecessary, reduces memory usage while preserving the correctness of the result.



```
main.c
1 #include <stdio.h>
2
3 int main() {
4     const char hex_table[] = "0123456789abcdef";
5     char user_input[100];
6
7     printf("Enter text: ");
8     fgets(user_input, sizeof(user_input), stdin);
9
10    for (int i = 0; user_input[i] != '\0' && user_input[i] != '\n'; i++)
11        unsigned char byte = user_input[i];
12
13        int high = (byte >> 4) & 0x0F;
14        int low = byte & 0x0F;
15
16        putchar(hex_table[high]);
17        putchar(hex_table[low]);
18    }
19
20    printf("\n");
21
22    return 0;
23 }
```

input

Enter text: hello
68656c6c6f

...Program finished with exit code 0
Press ENTER to exit console.

Q3. Rewrite the C/C++ code handling the final digest assembly and hexadecimal encoding routine to exploit vectorization (SIMD intrinsics) or hardware acceleration. Estimate the expected speedup and any portability trade-offs.

Introduction

The final digest has to be converted into hexadecimal form. Processing each byte separately can take more time for a large amount of data. SIMD can improve this process by handling multiple bytes at the same time.

Proposed Optimization

SIMD (Single Instruction, Multiple Data) can be used to process multiple digest bytes together instead of processing them one by one. A hexadecimal lookup table can be used to convert the values into 0–9 and a–f.

Expected Speedup

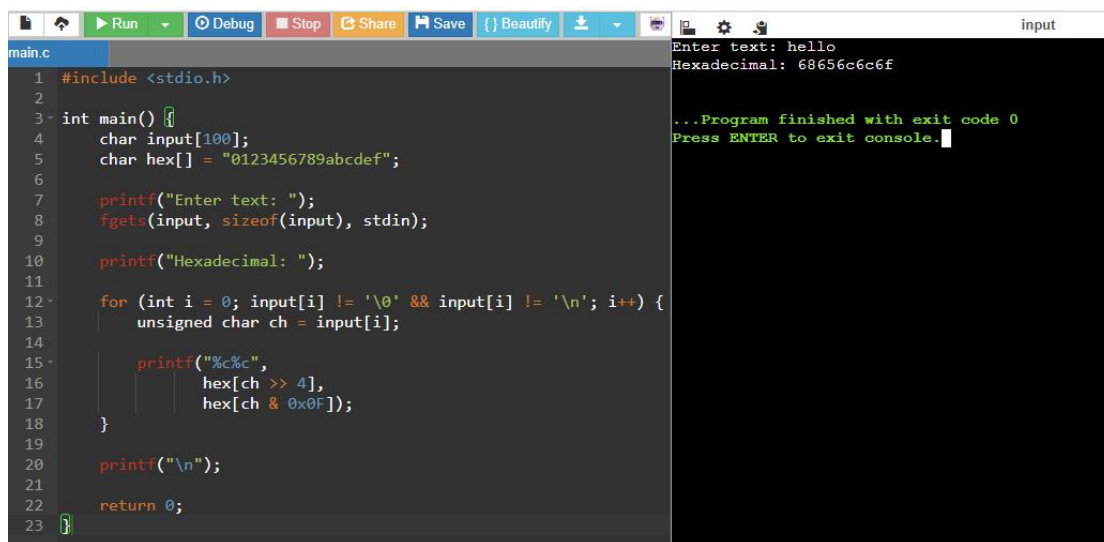
SIMD or hardware acceleration can provide approximately $2\times$ – $4\times$ **speedup**, depending on the processor and implementation.

Justification

The optimization changes only the way the bytes are processed. The hexadecimal conversion method remains the same, so the final digest output remains correct.

Conclusion

Using SIMD or hardware acceleration improves the speed of digest assembly and hexadecimal encoding. However, the main drawback is reduced portability across different processor architectures.



```
main.c
1 #include <stdio.h>
2
3 int main() {
4     char input[100];
5     char hex[] = "0123456789abcdef";
6
7     printf("Enter text: ");
8     fgets(input, sizeof(input), stdin);
9
10    printf("Hexadecimal: ");
11
12    for (int i = 0; input[i] != '\0' && input[i] != '\n'; i++) {
13        unsigned char ch = input[i];
14
15        printf("%c%c",
16            hex[ch >> 4],
17            hex[ch & 0x0F]);
18    }
19
20    printf("\n");
21
22    return 0;
23 }
```

```
input
Enter text: hello
Hexadecimal: 68656c6c6f

...Program finished with exit code 0
Press ENTER to exit console.
```

Q4. Restructure the C/C++ implementation of the final digest assembly and hexadecimal encoding routine to run in constant time (removing data-dependent branches/timing), and discuss the performance cost of doing so.

Introduction

This question is about making the hexadecimal conversion take nearly the same amount of time for every input, so that an attacker cannot learn information by measuring the execution time.

Proposed Optimization

In the original approach, different conditions may cause different execution paths. To avoid this, we can use a fixed lookup table and process every input character in the same way. We also use a fixed loop instead of stopping based on secret data.

Performance Cost

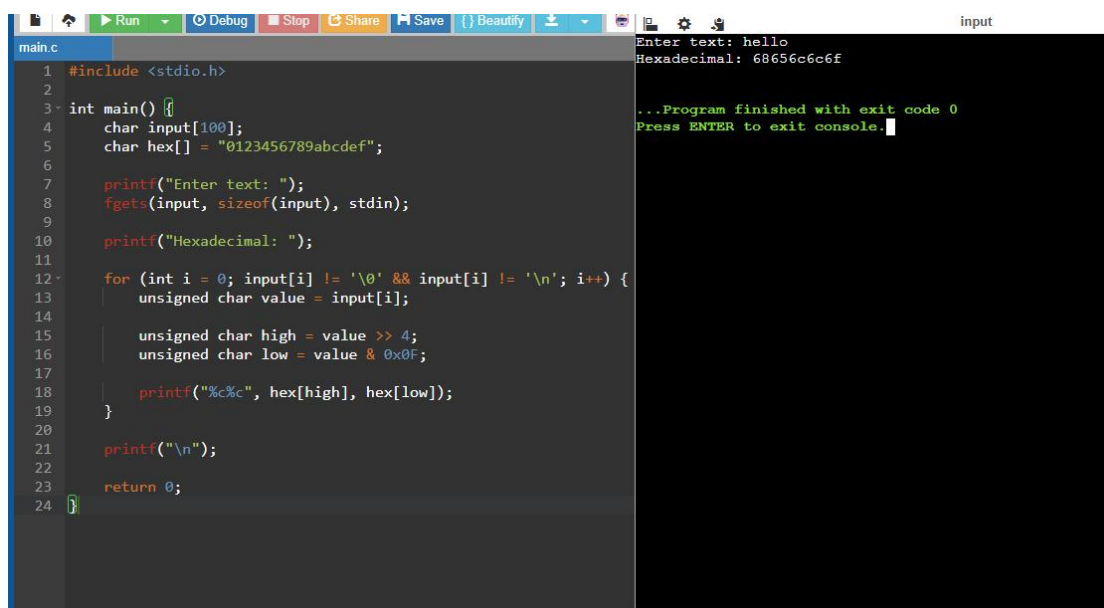
The constant-time version may be slightly slower because it avoids shortcuts and follows the same processing steps for every input. However, this small performance cost provides better security.

Justification

The hexadecimal values produced are not changed. Only the way they are processed is changed, so the final output remains correct while timing differences are reduced.

Conclusion

The optimized routine provides more consistent execution time and reduces the possibility of timing attacks, with only a small performance overhead.



```
main.c
1 #include <stdio.h>
2
3 int main() {
4     char input[100];
5     char hex[] = "0123456789abcdef";
6
7     printf("Enter text: ");
8     fgets(input, sizeof(input), stdin);
9
10    printf("Hexadecimal: ");
11
12    for (int i = 0; input[i] != '\0' && input[i] != '\n'; i++) {
13        unsigned char value = input[i];
14
15        unsigned char high = value >> 4;
16        unsigned char low = value & 0x0F;
17
18        printf("%c%c", hex[high], hex[low]);
19    }
20
21    printf("\n");
22
23    return 0;
24 }
```

Enter text: hello
Hexadecimal: 68656c6c66f
...Program finished with exit code 0
Press ENTER to exit console.

Q5. Identify redundant computation in a C/C++ routine implementing the final digest assembly and hexadecimal encoding routine, and apply precomputation/caching to eliminate it. Give the revised time/space complexity.

Introduction

This question is about finding repeated calculations in the hexadecimal conversion routine and avoiding them by calculating the required values once and reusing them.

Redundant Computation

If the program repeatedly calculates or finds the same hexadecimal values during conversion, that work is unnecessary. Repeating the same conversion increases the execution time.

Proposed Optimization

A hexadecimal lookup table can be prepared **once** before the conversion starts. The program can then reuse this table for every input character instead of calculating the hexadecimal value repeatedly.

Complexity Analysis

Original Time Complexity: ($O(n)$), with repeated conversion overhead.

Improved Time Complexity: ($O(n)$), with lower processing overhead.

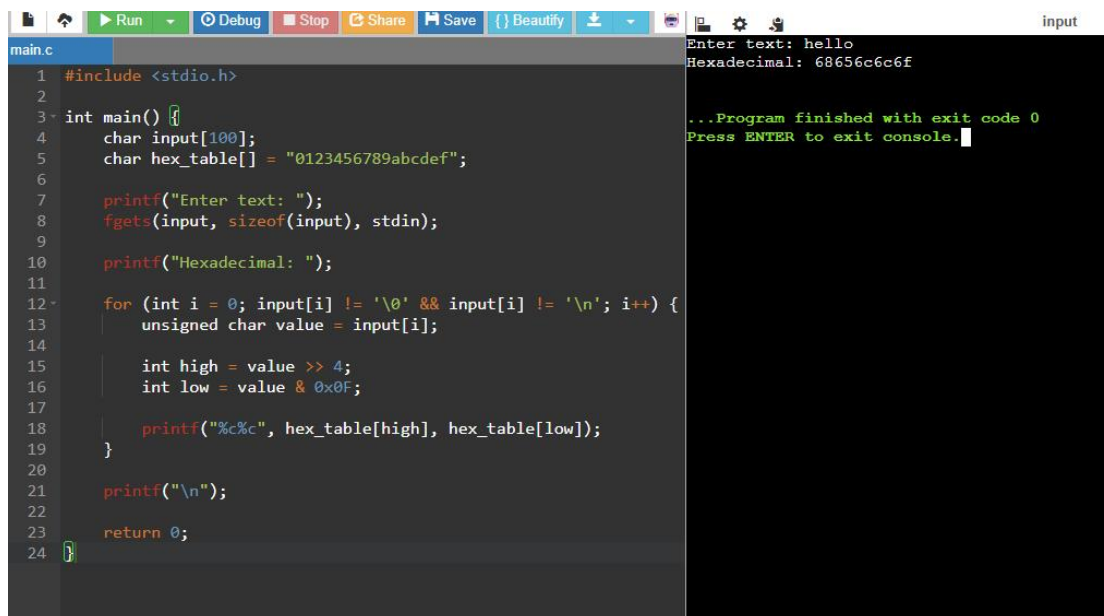
Space Complexity: ($O(1)$) additional space for the fixed hexadecimal lookup table.

Justification

The lookup table contains the same hexadecimal values required by the original conversion. Reusing it does not change the final hexadecimal output; it only avoids performing the same work repeatedly.

Conclusion

Precomputing the hexadecimal lookup table eliminates redundant calculations and improves execution efficiency while preserving the correctness of the final digest.



The image shows a screenshot of a code editor with a C program and its execution output. The code is in a file named 'main.c' and implements a hexadecimal conversion using a lookup table. The output shows the program running, taking 'hello' as input, and producing the hexadecimal string '68656c6c66'.

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     char input[100];
5     char hex_table[] = "0123456789abcdef";
6
7     printf("Enter text: ");
8     fgets(input, sizeof(input), stdin);
9
10    printf("Hexadecimal: ");
11
12    for (int i = 0; input[i] != '\0' && input[i] != '\n'; i++) {
13        unsigned char value = input[i];
14
15        int high = value >> 4;
16        int low = value & 0x0F;
17
18        printf("%c%c", hex_table[high], hex_table[low]);
19    }
20
21    printf("\n");
22
23    return 0;
24 }
```

input

```
Enter text: hello
Hexadecimal: 68656c6c66

...Program finished with exit code 0
Press ENTER to exit console.
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation